

MODEL CONTEXT PROTOCOL

COMPRENDRE, CONSTRUIRE, TESTER

De la théorie du protocole à un serveur qui tourne sur votre machine. Un guide de terrain, écrit à partir d'un serveur réellement construit et débogué.

Guide pratique · Version 1.0
SDK Python mcp 1.30 · Transport stdio · MCP Inspector v2
Octobre 2026

SOMMAIRE

Ce que couvre ce guide

01	Le problème que MCP résout	3
02	Architecture et transports	4
03	Les trois primitives	6
04	Construire un serveur, pas à pas	8
05	Exposer des resources	10
06	Les prompts : la couche utilisateur	11
07	Côté client : la boucle agentique	12
08	Tester : les trois niveaux	13
09	Étude de cas : un serveur documentaire	15
10	Les pièges qui coûtent des heures	20
11	Checklist de mise en production	22
12	Aller plus loin	24

Comment lire ce guide

Les trois premiers chapitres sont conceptuels : ils expliquent pourquoi MCP existe et comment le protocole est structuré. Les chapitres 4 à 9 sont opérationnels : vous tapez les commandes, vous obtenez un serveur qui répond. Les chapitres 10 et 11 sont ceux qu'on ne trouve dans aucun tutoriel : les erreurs réelles, et comment les reconnaître en dix secondes.

CHAPITRE 01

Le problème que MCP résout

Avant : l'intégration artisanale

Vous voulez que Claude lise vos fichiers, interroge votre base, envoie des messages dans Slack. Chaque intégration demande du travail spécifique : décrire l'API à l'attention du modèle, écrire un schéma JSON à la main, coder l'appel, gérer les erreurs, répéter pour chaque outil et chaque application cliente. Trois fournisseurs de modèles × cinq sources de données = quinze intégrations à maintenir.

Le vrai coût n'est même pas le code : c'est le **schéma**. Pour qu'un modèle sache appeler un outil, il faut lui décrire précisément son nom, son rôle, ses paramètres, leurs types et leurs contraintes. Un schéma écrit à la main dérive silencieusement du code réel, et le modèle se met à appeler le mauvais outil avec les mauvais arguments.

Après : un protocole, des serveurs interchangeables

MCP standardise le dialogue entre une application qui héberge un modèle (le **client**, ou host) et un programme qui expose des capacités (le **serveur** MCP). Un serveur écrit une fois fonctionne avec n'importe quel client compatible : Claude Desktop, Claude Code, votre propre application.

L'analogie la plus juste est celle du **Language Server Protocol** pour les éditeurs de code. Avant LSP, chaque éditeur réimplémentait le support de chaque langage. Après LSP, un serveur de langage sert tous les éditeurs. MCP fait la même chose pour les capacités des modèles.

> MCP ne fait pas « parler le modèle à votre base de données ». Il déplace la responsabilité de la description et de l'exécution des outils **du client vers un serveur spécialisé**. Le client devient générique ; le serveur porte la connaissance métier.

Ce que MCP n'est pas

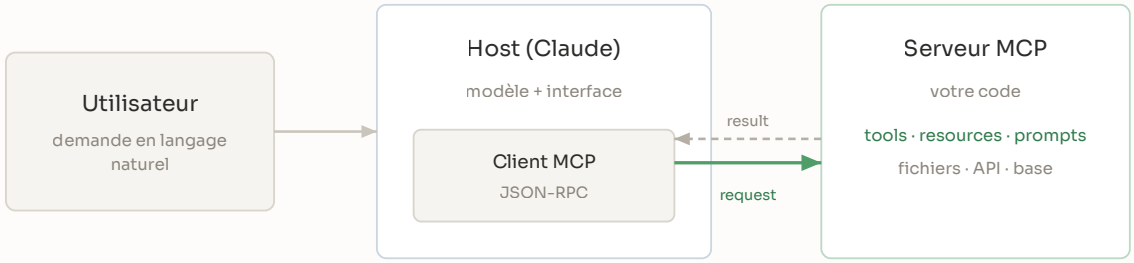
- Ce n'est pas un format d'API REST : c'est un protocole de session, avec négociation de capacités et appel de méthodes.
- Ce n'est pas un ORM ni un cadre d'agent : il ne décide pas quand appeler un outil, il fournit le canal et le contrat.
- Ce n'est pas lié à un fournisseur de modèle : c'est un standard ouvert.

CHAPITRE 02

Architecture et transports

Les trois rôles

RÔLE	RESPONSABILITÉ	EXEMPLES
Host	L'application qui héberge le modèle et l'interface utilisateur. Elle décide quels serveurs exposer et arbitre les permissions.	Claude Desktop, Claude Code, une app maison
Client	Le composant qui maintient une connexion à un serveur. Un host en ouvre un par serveur déclaré.	Objet `ClientSession` du SDK
Server	Le programme qui expose tools, resources et prompts, et exécute le travail.	Votre `mcp_server.py`



Un host, un client par serveur, des serveurs spécialisés.

Deux transports, un même protocole

Le protocole est **agnostique du transport** : les messages sont les mêmes, seul le tuyau change.

TRANSPORT	FONCTIONNEMENT	QUAND L'UTILISER
stdio	Le client lance le serveur en sous-processus et dialogue par l'entrée et la sortie standard. Pas de réseau, pas de port, pas d'authentification.	Serveur local : fichiers, outils de dev, Claude Desktop
Streamable HTTP	Le serveur écoute sur une URL. Sessions, en-têtes, streaming des réponses.	Serveur distant, partagé, multi-utilisateurs

> **Conséquence directe du stdio**
>
> En stdio, le serveur n'affiche rien et ne s'arrête jamais : il attend des messages sur son entrée standard. Un terminal qui semble figé après `mcp run mcp_server.py` n'est pas un bug, c'est le fonctionnement normal. Corollaire : tout ce que vous écrivez sur la sortie standard corrompt le protocole. Journalisez sur `stderr` .

Le dialogue, en clair

1. **initialize** — négociation : version du protocole, capacités des deux côtés, informations sur le serveur.
2. **notifications/initialized** — le client confirme.
3. **Découverte** — `tools/list`, `resources/list`, `prompts/list`.
4. **Usage** — `tools/call`, `resources/read`, `prompts/get`.
5. **Fermeture** — fin du processus ou de la session HTTP.

CHAPITRE 03

Les trois primitives

MCP expose trois familles de capacités. La différence n'est pas technique : elle tient à **qui décide** de les utiliser.

PRIMITIVE	QUI DÉCIDE	ANALOGIQUE À	SERT À
Tool	le modèle	Un appel de fonction, un POST	Agir : créer, modifier, calculer, appeler une API
Resource	l'application	Un GET, une URI	Lire des données, alimenter un contexte, un autocomplete
Prompt	l'utilisateur	Une commande slash, un template	Fournir des instructions de haute qualité, pré-écrites

> Ça lit des données → **resource**. Ça agit ou ça peut échouer → **tool**. C'est une consigne destinée au modèle → **prompt**.

Pourquoi la distinction compte

- **Coût** — un tool call est une décision du modèle, donc un aller-retour de tokens. Une resource lue par l'application ne coûte rien en raisonnement.
- **Fiabilité** — pour injecter le contenu d'un fichier mentionné avec `@fichier`, passer par une resource garantit que le contenu arrive. Passer par un tool dépend de la bonne volonté du modèle.
- **Contrôle** — les resources sont typiquement soumises à l'approbation de l'utilisateur ; les tools peuvent être restreints côté host.

Anatomie d'un tool

```python
@mcp.tool(
    name="read_doc_contents",
    description="Read the contents of a document and return it as a string.",
)
def read_document(
    document_id: str = Field(description="Id of the document to read"),
) -> str:
    if document_id not in docs:
        raise ValueError(f"Doc with id {document_id} not found")
    return docs[document_id]
```

ÉLÉMENT	CE QU'IL DEVIENT CÔTÉ PROTOCOLE
`@mcp.tool`	Enregistre la fonction et génère le JSON Schema
Le type hint `str`	`"type": "string"` dans le schéma
`Field(description=…)`	La description du paramètre. C'est du **prompt engineering**, pas de la décoration
`description=`	Ce que le modèle lit pour choisir quel outil appeler
`raise ValueError`	Un résultat d'erreur renvoyé au modèle, pas un crash

> **Le gain principal du SDK**
>
> Vous n'écrivez jamais de schéma JSON à la main. Le schéma est dérivé du code : il ne peut pas dériver de l'implémentation. C'est la raison d'être des décorateurs.

Le schéma, tel qu'il circule vraiment

```
{"properties": {
    "document_id": {
      "description": "Id of the document to read",
      "title": "Document Id",
      "type": "string"
    }
  },
 "required": ["document_id"],
 "title": "read_documentArguments",
 "type": "object"}
```

Remarquez `title` : le SDK dérive aussi les titres lisibles depuis les noms de paramètres. Rien de tout cela n'a été écrit à la main.

CHAPITRE 04

Construire un serveur, pas à pas

Étape 1 — l'environnement

```
# avec uv (recommandé)
uv venv
uv pip install "mcp[cli]"

# ou avec pip
python3 -m venv .venv
.venv/bin/pip install "mcp[cli]"
```

Piège de version

Le SDK est passé en version 2.x, où `FastMCP` a été renommé `MCPServer` et où plusieurs signatures ont changé. Tout le code écrit pour la version 1.x casse avec `No module named 'mcp.server.fastmcp'`. Épinglez la version utilisée par votre tutoriel : `pip install "mcp[cli]<2"`.

Étape 2 — le squelette

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("DocumentMCP", log_level="ERROR")

# ... tools, resources, prompts ici ...

if __name__ == "__main__":
    mcp.run()          # transport stdio par défaut
```

C'est un serveur MCP complet et valide. Trois lignes. Le nom passé au constructeur est celui que verront les clients lors de l'initialisation.

Étape 3 — un état applicatif

Pour l'exemple, un simple dictionnaire en mémoire suffit. Dans un vrai serveur, ce serait une base, une API, un dossier de fichiers.

```
docs = {
    "plan.md": "# Project Plan\n\n1. Gather requirements\n2. Build the MCP server",
    "report.pdf": "Quarterly Report - Q3\n\nRevenue grew by 12%...",
}
```

Étape 4 — les tools

```python
from pydantic import Field

@mcp.tool(name="read_doc_contents", description="Read a document's contents.")
def read_document(document_id: str = Field(description="Id of the document")) -> str:
    if document_id not in docs:
        raise ValueError(f"Doc with id {document_id} not found")
    return docs[document_id]


@mcp.tool(name="edit_document", description="Replace a string in a document.")
def edit_document(
    document_id: str = Field(description="Id of the document to edit"),
    old_string: str = Field(description="Exact text to replace, whitespace included"),
    new_string: str = Field(description="Replacement text"),
) -> str:
    if document_id not in docs:
        raise ValueError(f"Doc with id {document_id} not found")
    if old_string not in docs[document_id]:
        raise ValueError(f'Text "{old_string}" not found in document')
    docs[document_id] = docs[document_id].replace(old_string, new_string, 1)
    return "Document edited successfully"
```

Trois détails qui font la différence

`replace(…, 1)` : une seule occurrence, car un remplacement global est une source de bugs silencieux. · La description de `old_string` précise « whitespace included » : le modèle comprend qu'il doit recopier exactement. · Les erreurs sont explicites et actionnables : le modèle peut les lire et corriger son appel au tour suivant.

Étape 5 — un tool qui n'écrit pas

```python
@mcp.tool(name="search_docs", description="Search a keyword across all documents.")
def search_docs(keyword: str = Field(description="Keyword to look for")) -> dict[str, str]:
    hits = {}
    for doc_id, content in docs.items():
        if keyword.lower() in content.lower():
            i = content.lower().index(keyword.lower())
            hits[doc_id] = "..." + content[max(0, i - 40): i + 60] + "..."
    return hits
```

Le type de retour `dict[str, str]` est sérialisé automatiquement en JSON. Pas de `json.dumps`, pas de conversion manuelle : le SDK s'en charge.

Étape 6 — lancer

```
python mcp_server.py        # stdio : pour un client
mcp dev mcp_server.py       # inspecteur web : pour vous
```

CHAPITRE 05

Exposer des resources

Une resource se comporte comme un handler de GET : on lui donne une URI, elle renvoie des données. Il en existe deux formes.

Resource directe : une URI fixe

```python
@mcp.resource("docs://documents", mime_type="application/json")
def list_docs() -> list[str]:
    return list(docs.keys())
```

Aucun paramètre. Parfaite pour lister ce qui est disponible : c'est exactement ce qui alimente un autocomplete quand l'utilisateur tape `@`.

Resource templatée : des paramètres dans l'URI

```python
@mcp.resource("docs://documents/{doc_id}", mime_type="text/plain")
def fetch_doc(doc_id: str) -> str:
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")
    return docs[doc_id]
```

Le SDK extrait `{doc_id}` du gabarit d'URI et le passe en argument nommé. Vous n'écrivez aucun parseur.

Les URI : un simple espace de noms

- Le préfixe (`docs://`) est **libre**. Ce n'est pas une URL réseau, c'est un identifiant de schéma.
- Choisissez un préfixe qui décrit votre domaine : `db://` , `git://` , `notes://` .
- Le `mime_type` indique au client comment interpréter la réponse : `application/json` pour des structures, `text/plain` pour du texte, `application/pdf` pour du binaire.

Ressources ou tools ? Le cas du `@`

Quand l'utilisateur tape `@report.pdf` dans une interface de chat, deux opérations se cachent :

1. Lister les documents disponibles pour l'autocomplete → **resource directe**.
2. Récupérer le contenu du document mentionné pour l'injecter dans le prompt → **resource templatée**.

Résultat : le document arrive dans le contexte **sans qu'aucun tool ne soit appelé**. Plus rapide, moins coûteux, déterministe.

CHAPITRE 06

Les prompts : la couche utilisateur

Un prompt MCP est un template d'instructions exposé par le serveur et déclenché par l'utilisateur, généralement via une commande slash. Il ne s'exécute pas tout seul : c'est l'humain qui le choisit.

```
@mcp.prompt(
    name="format",
    description="Rewrite the document as clean markdown.",
)
def format_document(
    doc_id: str = Field(description="Id of the document to format"),
) -> str:
    return f"""Please rewrite the following document into clean,
well-structured markdown. Use headings, bullet points and emphasis
where it improves readability. Do not invent information.

<document id="{doc_id}">
{docs.get(doc_id, "Document not found")}
</document>"""
```

Un prompt sert à trois choses :

- **Encapsuler une expertise** — la bonne formulation, une fois, au bon endroit, plutôt que dans la tête de chaque utilisateur.
- **Standardiser** — toute l'équipe produit le même format de document.
- **Découvrir** — le client liste les prompts disponibles, ce qui rend les workflows visibles dans l'interface.

> **Où vit la logique ?**
>
> Le prompt est calculé côté serveur. C'est ce qui permet d'y injecter des données, d'appliquer des règles métier et de faire évoluer la formulation sans toucher au client.

CHAPITRE 07

Côté client : la boucle agentique

Un client MCP tient en quelques lignes avec le SDK. Le schéma de connexion est toujours le même.

```python
import sys, anyio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    params = StdioServerParameters(command=sys.executable, args=["mcp_server.py"])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            info = await session.initialize()

            # découverte
            for t in (await session.list_tools()).tools:
                print(t.name, t.inputSchema)

            # usage
            res = await session.call_tool("read_doc_contents", {"document_id": "plan.md"})
            print(res.content[0].text)

anyio.run(main)
```

Le double `async with` n'est pas décoratif : le premier gère le sous-processus, le second la session protocolaire. Fermer la session envoie la séquence d'arrêt au serveur.

Brancher le modèle

Le pont avec l'API Anthropic est direct, car le JSON Schema produit par le SDK MCP est déjà au format attendu :

```python
def to_anthropic_tools(tools):
    return [{"name": t.name, "description": t.description,
             "input_schema": t.inputSchema} for t in tools]

# boucle : tant que le modèle demande un tool, on exécute et on renvoie
while True:
    response = await client.messages.create(
        model=MODEL, max_tokens=4096,
        tools=anthropic_tools, messages=messages,
    )
    messages.append({"role": "assistant", "content": response.content})
    if response.stop_reason != "tool_use":
        break
```

> **Deux règles d'or de la boucle**
>
> Renvoyez un `tool_result` pour chaque `tool_use`, y compris en cas d'erreur : un bloc orphelin fait échouer le tour suivant. Et renvoyez l'exception serveur avec `is_error: true` plutôt que de laisser planter la boucle — c'est ainsi que le modèle se corrige.

CHAPITRE 08

Tester : les trois niveaux

Un serveur MCP se teste à trois niveaux, du plus mécanique au plus réaliste. Franchissez-les dans l'ordre : chaque niveau isole une catégorie de bugs.

Niveau 1 — le test de protocole, sans clé API

```python
async def main():
    params = StdioServerParameters(command=sys.executable, args=["mcp_server.py"])
    async with stdio_client(params) as (r, w):
        async with ClientSession(r, w) as s:
            await s.initialize()

            # 1. les tools sont-ils annoncés ?
            tools = (await s.list_tools()).tools
            assert {t.name for t in tools} == {
                "read_doc_contents", "edit_document", "search_docs"}

            # 2. un appel nominal fonctionne-t-il ?
            res = await s.call_tool("read_doc_contents", {"document_id": "plan.md"})
            assert "Project Plan" in res.content[0].text

            # 3. la ressource directe répond-elle en JSON ?
            assert (await s.read_resource("docs://documents")).contents[0].text

            # 4. la ressource templatée résout-elle son paramètre ?
            await s.read_resource("docs://documents/report.pdf")

            # 5. le prompt est-il récupérable ?
            await s.get_prompt("format", {"doc_id": "email.md"})
```

Niveau 2 — l'inspecteur

```
uv run --with mcp mcp dev mcp_server.py
```

La commande lance un serveur proxy et une interface web. Le terminal affiche une URL contenant un jeton d'authentification :

```
http://localhost:6274/?MCP_PROXY_AUTH_TOKEN=8f2c...
```

Ouvrez **l'URL complète**. Sans le jeton, la connexion est refusée. Laissez le terminal ouvert : c'est lui qui exécute le serveur.

- **Barre latérale** — les tools, resources et prompts exposés. « Resources » contient les ressources directes ; « Resource Templates » les ressources paramétrées.
- **Zone centrale** — le formulaire d'appel et le résultat, y compris le « Structured Output » (le JSON brut).
- **Panneau droit** — l'onglet « Protocol » journalise chaque message JSON-RPC avec son code de retour et sa durée.

> **Le test le plus instructif : provoquer une erreur**
>
> Appelez `edit_document` avec un `old_string` qui n'existe pas. Vous obtenez une erreur lisible et la session reste connectée : c'est la preuve que vos exceptions deviennent des résultats d'erreur exploitables par le modèle, et non des crashs.

Niveau 3 — un modèle aux commandes

Le seul niveau qui valide la vraie question : le modèle choisit-il le bon outil, seul ? Il faut une clé API, et trois questions suffisent.

QUESTION POSÉE	COMPORTEMENT ATTENDU	PRIMITIVE
`Quels documents parlent de revenue ?`	appelle `search_docs` de lui-même	tool
`@report.pdf résume les chiffres`	**aucun** tool call, contenu injecté	resource
`/format email.md`	instructions récupérées puis envoyées	prompt

La deuxième ligne est la plus importante : si un tool call apparaît là où une resource devrait suffire, votre architecture est mal découpée.

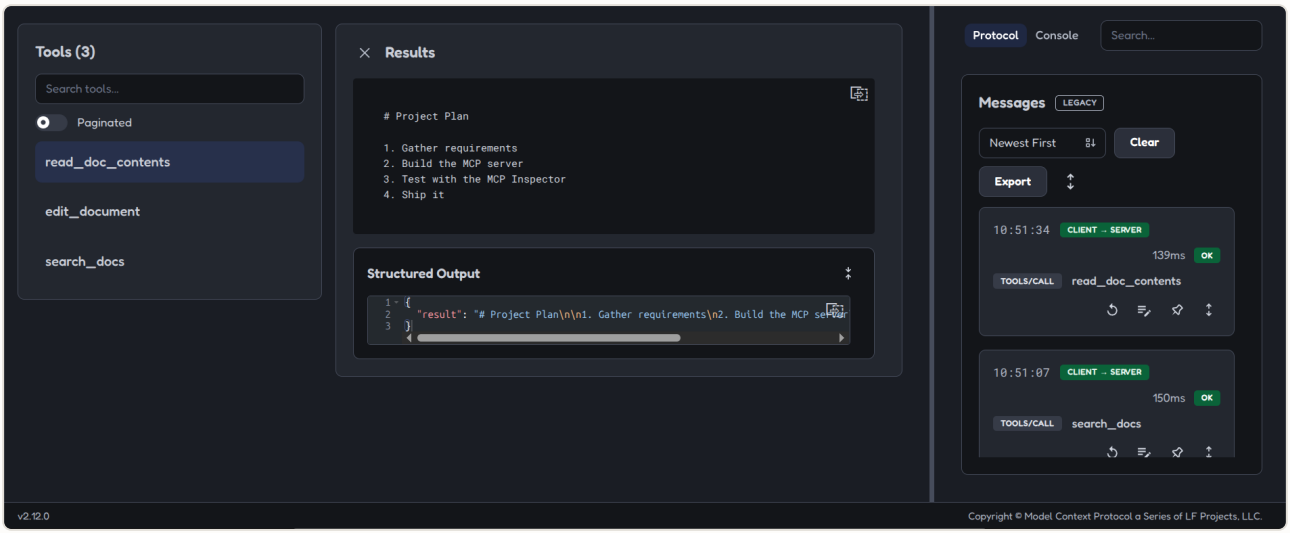
CHAPITRE 09

Étude de cas : un serveur documentaire

Le projet complet — `mcp_server.py`, `mcp_client.py`, `test_server.py` — tient en trois fichiers. Cette section montre les preuves d'exécution obtenues en le testant.

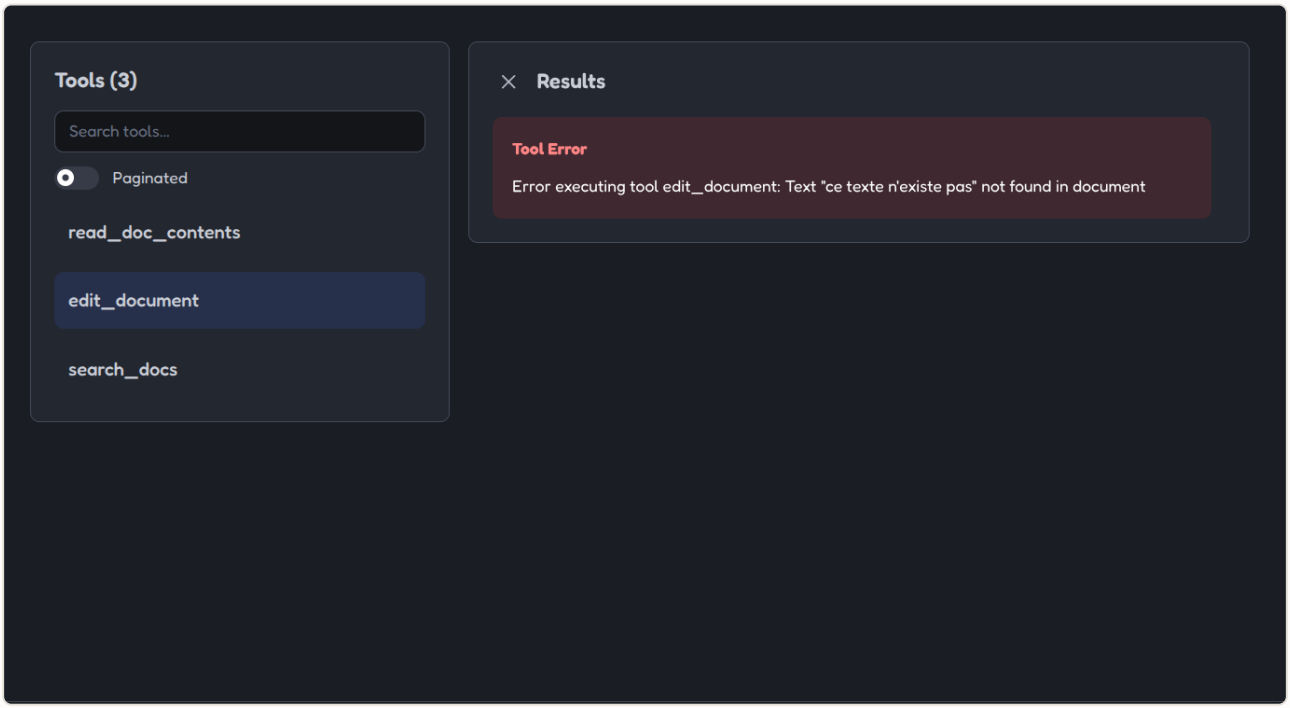
ÉLÉMENT	TYPE	RÔLE
`read_doc_contents`	tool	lire un document
`edit_document`	tool	remplacer une chaîne, une seule occurrence
`search_docs`	tool	rechercher un mot-clé, avec extrait
`docs://documents`	resource directe	liste des ids, JSON, pour l'autocomplete
`docs://documents/{doc_id}`	resource templatée	contenu d'un document, texte brut
`format`	prompt	réécriture en markdown propre

Preuve n°1 — les tools sont annoncés avec leur schéma



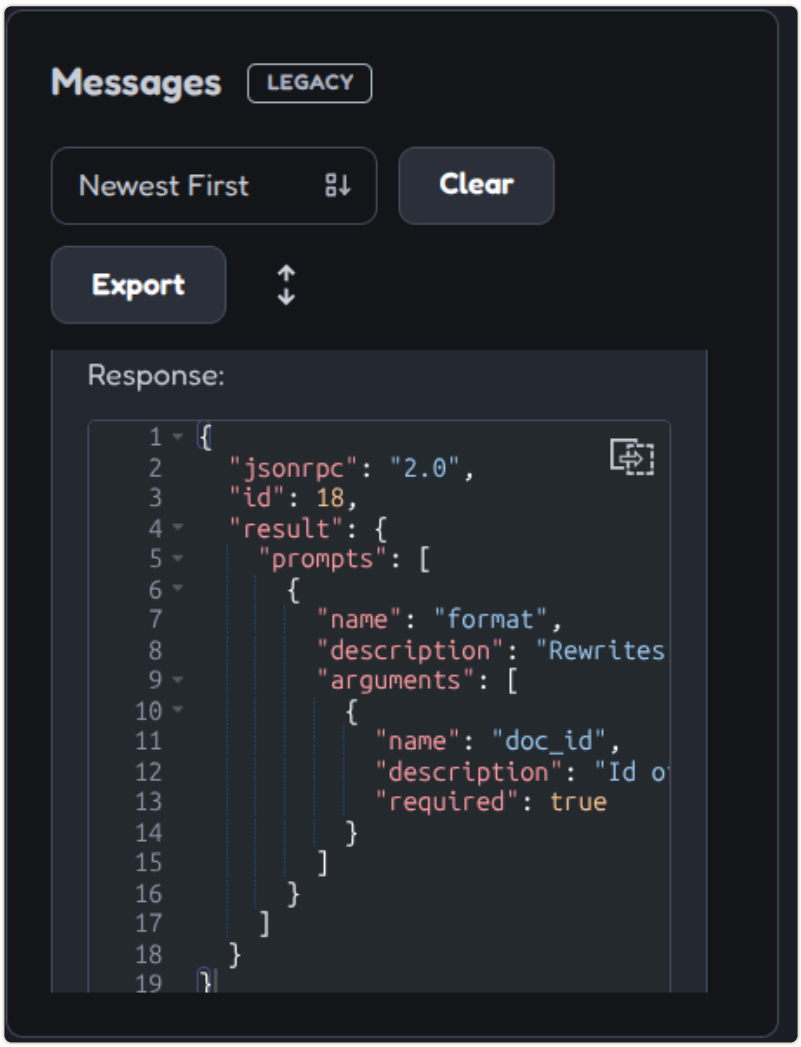
À gauche, les trois tools annoncés ; au centre, le résultat d'un appel et le « Structured Output » (JSON brut) ; à droite, le journal des appels avec leurs durées. Le schéma de chaque paramètre est dérivé du code Python, descriptions incluses.

Preuve n°2 — une erreur reste une réponse, pas un crash



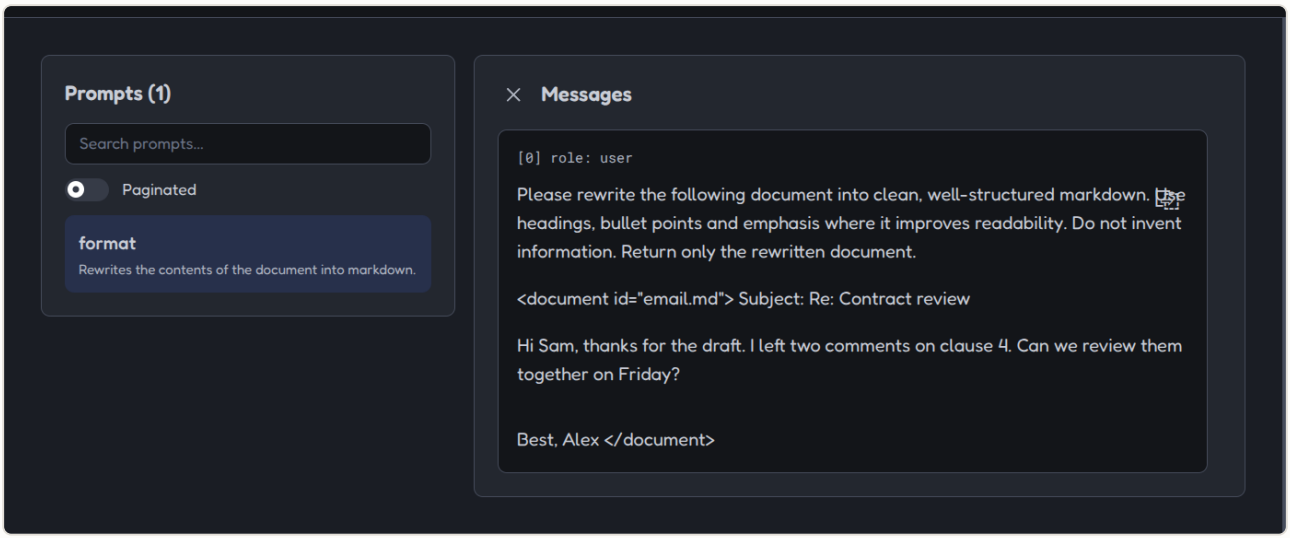
« Tool Error » : le message est explicite et la session survit. Le modèle peut relire l'erreur et reformuler son appel.

Preuve n°3 — le protocole à nu



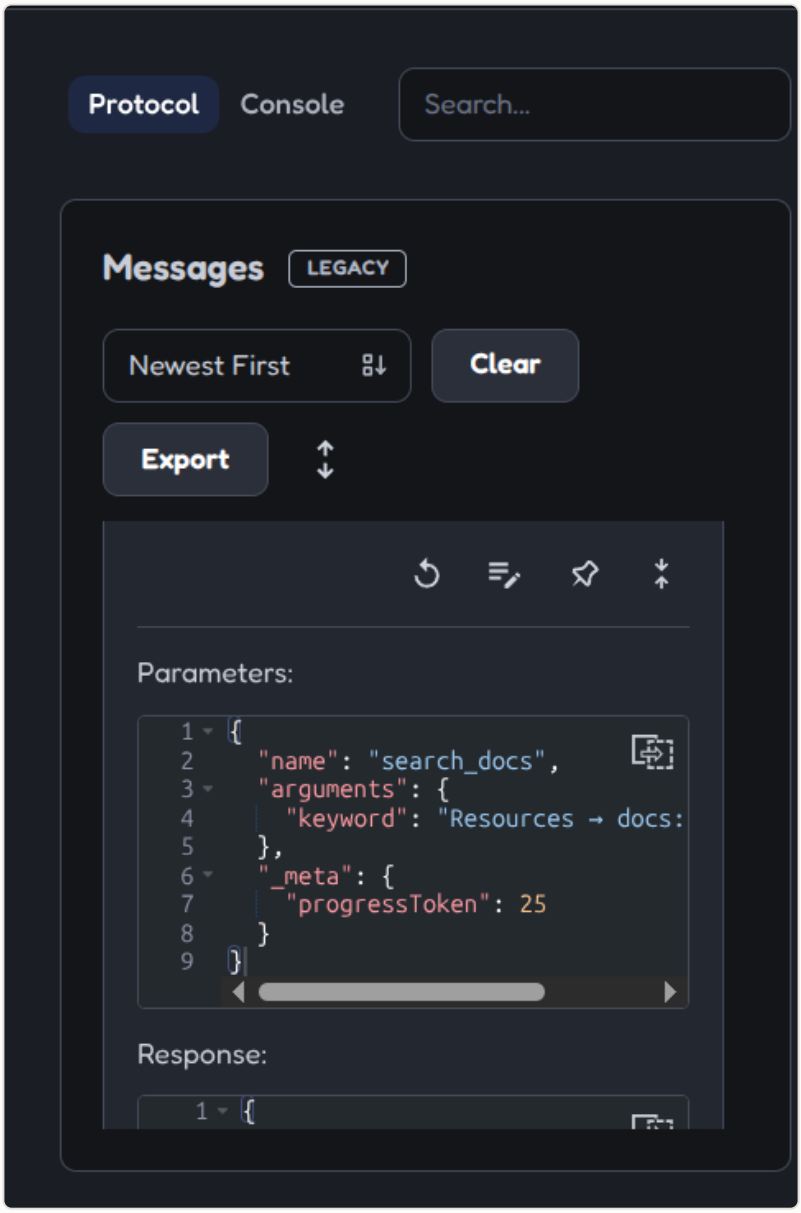
Journal JSON-RPC : `"jsonrpc": "2.0"`, `"id": 18`, un `result` contenant `prompts` avec leurs `arguments` et `required`. C'est exactement ce que la spécification décrit, et ce que le client transmet au modèle.

Preuve n°4 — les prompts sont calculés côté serveur



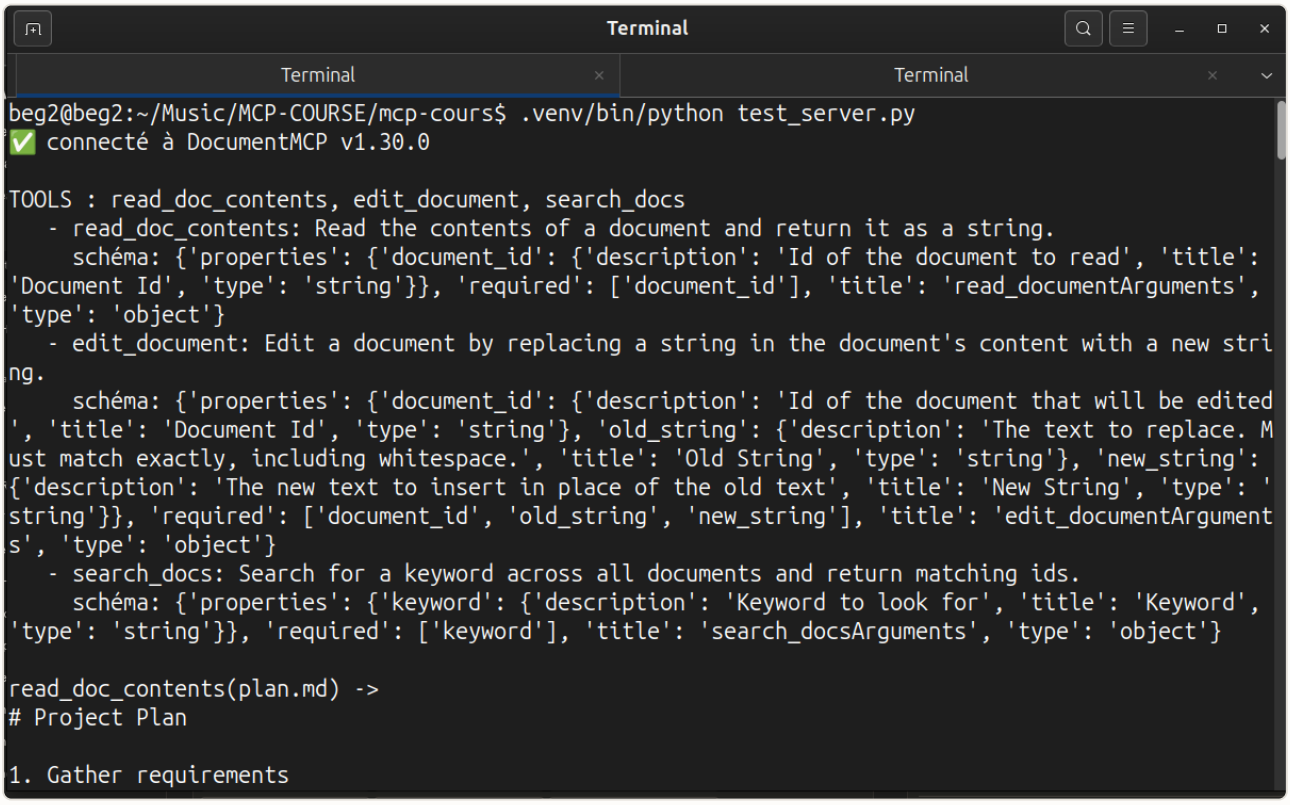
Le prompt `format` reçoit `doc_id` et construit le message complet, balise `<document>` incluse, avec les instructions de réécriture.

Preuve n°5 — l'argument transmis au serveur



Le détail d'un `tools/call` : `search_docs` reçoit bien `{"keyword": "Resources → docs"}`, avec les métadonnées de progression ajoutées par le protocole. Le nom du tool et la forme des arguments sont exactement ceux que le schéma annonçait.

Preuve n°6 — la session est établie



Serveur connecté en transport stdio, négociation de version protocole réussie, appels `resources/read` sur `docs://documents` avec leur latence.

Ce que cette étude de cas démontre

- Les trois primitives coexistent dans un même serveur sans se confondre.
- Le schéma n'est jamais écrit à la main : il découle des annotations.
- Une erreur métier est un **résultat**, pas une exception non gérée.
- Toute la chaîne est vérifiable sans navigateur et sans clé API.

CHAPITRE 10

Les pièges qui coûtent des heures

Cette section est le condensé des échecs réellement rencontrés en construisant ce serveur. Chaque entrée se présente sous la forme : symptôme, cause, solution.

SYMPTÔME	CAUSE RÉELLE	SOLUTION
`command not found: uv` juste après l'installation	Le binaire est installé dans `~/.local/bin`, absent du PATH (fréquent en fish)	`fish_add_path ~/.local/bin`, puis la même ligne dans `config.fish`
`syntax error: unexpected end of file` sur un `source`	Un copier-coller depuis un chat a transformé le chemin en lien markdown	Retaper la commande à la main ; ne jamais copier depuis un rendu riche
`No module named 'mcp'`	Le script est lancé avec le Python du système, pas celui du venv	`.venv/bin/python test_server.py` — chemin explicite, sans activation
`No module named 'mcp.server.fastmcp'`	SDK 2.x installé : `FastMCP` y est renommé `MCPServer`	`pip install "mcp[cli]<2"`
`Unable to determine which files to ship inside the wheel`	`uv run` voit un `pyproject.toml` et tente de builder le dossier comme un package	Ajouter `[tool.uv] package = false`, ou lancer `uv run --no-project …`, ou supprimer le `pyproject.toml`
Terminal figé, aucune sortie après `mcp run`	Comportement normal du transport stdio : le serveur attend un client	`mcp dev` pour un humain, `mcp run` pour un client
Trace `KeyboardInterrupt` au Ctrl+C	Interruption d'un processus bloqué en lecture sur stdin	Inoffensif. `pkill -f mcp_server.py` si besoin
`npx: command not found`	Node.js absent ; l'inspecteur en a besoin	Installer Node 20+ via le dépôt NodeSource, pas le `npm` d'apt
`nodejs : Conflicts: npm`	Le paquet `npm` de la distribution entre en conflit avec un nodejs NodeSource	Ne pas installer `npm` via apt : NodeSource le fournit déjà
Inspecteur « Disconnected »	URL ouverte sans le jeton `?MCP_PROXY_AUTH_TOKEN=…`	Cliquer sur le lien complet affiché dans le terminal
Le modèle n'appelle jamais l'outil attendu	La `description` dit ce que fait l'outil, pas quand l'utiliser	Décrire la situation d'usage, pas seulement l'action
Le modèle passe un mauvais identifiant	`Field(description=…)` imprécise, ou liste des valeurs absente	Énumérer les identifiants valides dans la description
Le serveur semble ne rien renvoyer	Un `print()` de debug pollue la sortie standard, qui porte le protocole	Journaliser sur `stderr` ou via le logger du SDK

> Face à un serveur qui « ne marche pas », faites toujours dans cet ordre : (1) le script de test répond-il ? (2) l'inspecteur exécute-t-il l'outil à la main ? (3) seulement ensuite, regarder les descriptions. Quatre-vingt-dix pour cent des problèmes se règlent à l'étape 1.

CHAPITRE 11

Checklist de mise en production

Contrat et descriptions

- Chaque tool a une description qui dit quand l'utiliser, pas seulement ce qu'il fait.
- Chaque paramètre a un `Field(description=…)` qui précise le format attendu et les contraintes.
- Les valeurs possibles sont énumérées quand la liste est finie.
- Les erreurs métier sont explicites et actionnables, jamais silencieuses.

Sécurité

- Toute entrée venant du modèle est validée : les tools sont une surface d'attaque, pas un formulaire de confiance.
- Les chemins sont normalisés et confinés à un répertoire racine (protection contre `../`).
- Les opérations destructrices sont explicites, réversibles si possible, ou soumises à confirmation.
- Aucun secret n'est journalisé ; les clés viennent de l'environnement.
- En transport HTTP : authentification, sessions, restriction des origines.

Robustesse

- Timeouts sur tous les appels sortants.
- Idempotence des écritures quand c'est possible.
- Le serveur ne meurt pas sur une erreur d'entrée : il renvoie une erreur.
- État partagé protégé si le serveur est concurrent.

Observabilité

- Journalisation structurée sur `stderr`, jamais sur `stdout`.
- Un horodatage et une durée par appel, pour repérer les tools lents.
- Un script de test automatisé exécutable en une commande, sans réseau.

Distribution

- Version épinglée du SDK (`mcp[cli]<2` par exemple).
- Un README qui donne la commande exacte de lancement.
- Déclaration dans les configs clients avec `--no-project` si le dossier contient un `pyproject.toml`.

Configuration client (Linux, Claude Desktop)

```
# ~/.config/Claude/claude_desktop_config.json
{
  "mcpServers": {
    "documents": {
      "command": "uv",
      "args": ["--no-project", "--with", "mcp", "mcp", "run",
               "/home/beg2/Music/MCP-COURSE/mcp-cours/mcp_server.py"]
    }
  }
}
```

Quittez complètement l'application puis relancez-la : le serveur n'est lu qu'au démarrage. Pour Claude Code, une seule commande : `claude mcp add documents -- python /chemin/mcp_server.py`.

CHAPITRE 12

Aller plus loin

Trois exercices progressifs

1. **Ajouter une resource de synthèse** `stats://documents` renvoyant, en JSON, le nombre de documents, le nombre total de mots et le document le plus long. Valide la sérialisation automatique et les types de retour composés.
2. **Ajouter un tool d'écriture complète** `create_document(document_id, content)` avec erreur si l'identifiant existe. Valide la gestion des conflits et les messages d'erreur destinés au modèle.
3. **Passer en transport HTTP** : `mcp.run(transport="streamable-http")`, puis connexion avec `mcp.client.streamable_http`. Valide la sortie du mono-utilisateur local.

Trois directions utiles

DIRECTION	CE QUE ÇA CHANGE
Persistance	Remplacer le dictionnaire par des fichiers ou une base. La surface MCP ne bouge pas : c'est le signe d'une bonne séparation.
Composition	Un serveur peut lui-même être client d'un autre serveur : découper par domaine plutôt que tout empiler.
Sécurité fine	Scoper les tools par rôle, journaliser les appels, exiger une confirmation sur les écritures.

Les trois idées à retenir

> 1 — **La primitive, c'est la politique.** Tool : le modèle décide. Resource : l'application décide. Prompt : l'utilisateur décide.
>
> 2 — **Le schéma est du code.** Les décorateurs et les annotations produisent le contrat ; les descriptions sont du prompt engineering. On ne les écrit pas deux fois.
>
> 3 — **Un serveur MCP se teste sans modèle.** Si le protocole répond et que l'inspecteur exécute correctement, le reste relève du paramétrage des descriptions.

Guide rédigé à partir d'un serveur MCP construit, testé et débogué de bout en bout (SDK Python `mcp 1.30`, transport stdio, MCP Inspector v2). Les captures d'écran proviennent de sessions réelles.